



RAPPORT FINAL

COURS 8INF924 INTERNET DES OBJETS

Sujet : Conception et implémentation d'une maison intelligente basée sur l'ESP32 et des capteurs IoT connectés à AWS

Rédigé par` : Eloi SEIDLITZ

Julot Déjabris Ngongang

Enseignant : Florentin Thullier, PhD

- 1. Introduction**
- 2. État de l'art et technologies utilisées**
- 3. Description de l'architecture matérielle**
- 4. Communication et infrastructure cloud**
- 5. Développement logiciel embarqué**
- 6. Intégration et tests**
- 7. Limites et perspectives**
- 8. Conclusion**
- 9. Annexes et bibliographie**

Introduction

Pendant plusieurs années, l'internet des Objets a transformé notre interaction avec notre environnement domestique. En connectant capteurs, actionneurs et systèmes d'informations à internet, les maisons intelligentes promettent une gestion énergétique optimisée, une amélioration du confort quotidien et une sécurité renforcée de ses occupants.

Malgré ces avancées, plusieurs défis sont connus. Notamment, l'automatisation des tâches quotidiennes sans compromission de la simplicité d'usage, l'amélioration du confort des occupants en réduisant la consommation énergétique, la confidentialité et l'intégrité des données échangées avec les systèmes embarqués afin de parer aux multiples attaques ciblant les objets connectés.

Notre objectif est de prototyper et déployer une architecture domotique minimaliste reposant sur un microcontrôleur de type ESP32, chargé de collecter les mesures de différents capteurs puis de les transmettre, via wifi, à la plateforme cloud AWS.

L'infrastructure cloud assurera le stockage, l'analyse en temps réel et la visualisation des données, tout en permettant le contrôle à distance des actionneurs (LED, Hauts Parleurs, relais). Notre architecture se veut reproductible, économiquement abordable et évolutive, afin de servir de base à des extensions futures.

Le code source complet de ce projet est disponible en open-source sur GitHub : <https://github.com/eloisd/smarthome>

Pour notre démarche projet, nous nous appuyerons sur un processus itératif en quatre étapes :

Prototypage matériel : il s'agit de la sélection, câblage et validation individuelle des capteurs et actionneurs autour de l'ESP32

Programmation embarquée : développement du firmware pour la collecte des données, la communication WebSocket et la gestion des événements locaux.

Intégration cloud : configuration des services AWS API Gateway, Lambda et DynamoDB pour l'ingestion, le stockage et le traitement des données.

Test et validation : tests de fonctionnalités et de résistance (charge, sécurité) afin d'évaluer la fiabilité, la latence et la robustesse de la solution.

II- État de l'art et technologie Utilisé

II-1 Maisons Intelligentes : Définitions, enjeux et cas d'usage

Une maison intelligente ou smart-home est un environnement résidentiel où les équipements (capteurs, actionneurs, électroménagers et système de sécurité) sont interconnectés et pilotés par un logiciel local ou hébergé dans un cloud. Le but d'un tel dispositif étant de délivrer un confort, optimiser la consommation énergétique et délivrer un environnement de sécurité tout en réduisant l'intervention humaine.

II-2 Technologie retenue pour le prototypage

ESP32 - Microcontrôleur central

SoC double-cœur 32 bits avec Wi-Fi/Bluetooth intégrés, 520 KB SRAM, coût <10 CAD. Collecte les données capteurs et communique directement avec le cloud sans passerelle.

AWS - Infrastructure cloud serverless

API Gateway WebSocket + Lambda offrent un canal bidirectionnel temps-réel avec mise à l'échelle automatique et facturation à l'usage, simplifiant le push d'événements cloud vers navigateur.

Communication réseau

Wi-Fi pour connectivité domestique, **WebSocket** pour tunnel full-duplex persistant sans surcoûts HTTP.

Capteurs et effecteurs

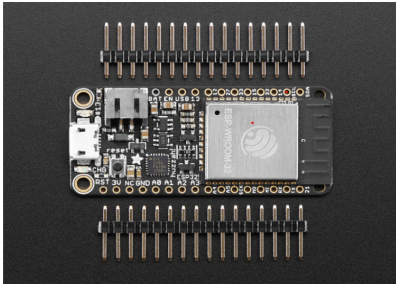
- **Tactile capacitif** (A700000006944498) : Contrôle tactile LED
- **Sonore** (DFR0034) : Détection bruit pour alertes
- **Ultrasonique** (SEN0307) : Mesure distance pour sécurité
- **LED** (DFR0021-B) : Retour visuel états système
- **Haut-parleur** (FIT0449) : Notifications sonores

Interface GPIO/I²C/SPI, alimentation 3.3V, gestion interruptions pour latence minimale.

III- Description de l'architecture Matérielle

III-1 présentation de l'ESP32 utilisé dans le cadre de notre projet

Nous nous sommes servis d'un **Adafruit HUZZAH32 – ESP32 Feather** qui est une carte de développement compacte et polyvalente basée sur le microcontrôleur ESP32 Wroom-32. Elle est de la gamme des Feather, conçue pour être légère, autonome et compatible avec des batteries LiPo rechargeables.

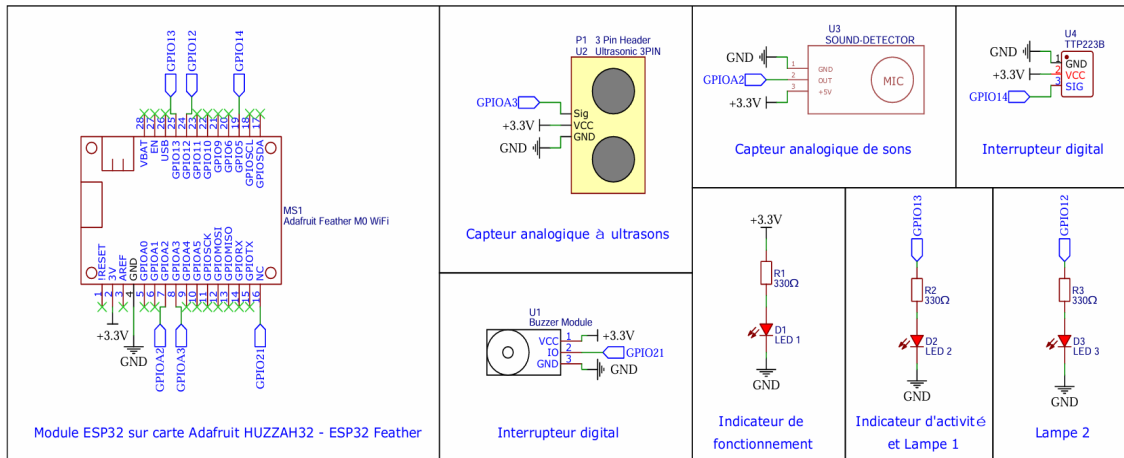


Microprocesseur : dual-core 32-bit Xtensa @240 Mhz
Connectivité : Wifi 802.11b/g/n et Bluetooth 4.2
Mémoire : 4Mo de flash
GPIO :20 Broches d'entrées/sorties numériques avec support ADC, DAC,PWM,I²C,SPI,UART
USB intégré : port micro-USB (programmation et alimentation)
Batterie : Connecteur JST pour LiPo 3.7V
Format Feather : compatible avec les Shields Feather Wing

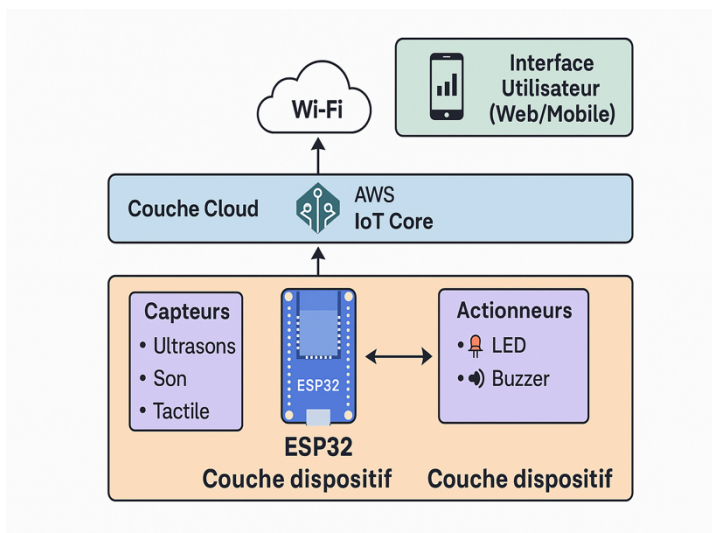
III-2 Description des Capteurs/ Effecteurs Connectés

Module	Interface/Pins	Fonctions et Notes d'integrations
Capteur tactile capacitif A700000006944498	Data: GPIO 4 (digital) Vcc: 3,3 V	Sert de bouton sans usure mécanique : un appui débite un front haut/bas que l'ESP32 interrompt pour allumer/éteindre la LED.
Capteur sonore analogique DFR0034	Out: GPIO 34 (ADC) Vcc : 3,3 V	Mesure le niveau SPL ; on fixe un seuil (≈ 60 dB) déclenchant une alerte ou l'allumage du haut-parleur.
Capteur ultrasonique SEN0307	Trigger: GPIO 12 Echo: GPIO 13 (via diviseur 2:1 ou level-shifter $\rightarrow$ 3,3 V) Vcc: 5 V	Calcule la distance (2 cm – 4 m) pour détection d'approche ou ouverture automatique ; l'écho 5 V doit être abaissé à 3,3 V pour protéger l'ESP32.
LED bleue DFR0021-B	Anode : GPIO 2 (PWM possible) Cathode : GND via résistance 220 Ω	Feedback visuel : statique, clignotement ou intensité modulée selon événements.
Haut-parleur FIT0449 (ampli intégré)	Audio In: GPIO 25 (DAC 1) Vcc: 5 V	Diffuse alertes sonores ou messages vocaux ; piloté par sorties DAC/PWM de l'ESP32.

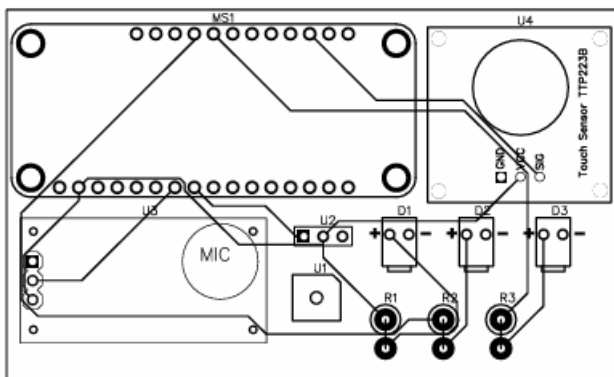
III-3 Schématique



III-4 Architecture



III-5 PCB



IV. Communication et infrastructure cloud

IV-1 Fonctionnement du WebSocket serverless AWS

Deux services managés sont proposés par AWS

Service	Usage Typique	Particularités Serverless
API Gateway (WebSocket API)	Push temps-réel générique (chat, commandes)	Gère : ouverture/fermeture de connexion, routage par « routes », intégration Lambda/DynamoDB. Aucune VM à gérer, la facturation est « pay-per-message »
AWS IoT Core (MQTT over WSS)	Télémétrie IoT, jumeaux d'objet	Même passerelle que MQTT natif mais sur port 443, conserve l'authN/autorisation basée sur certificats/politiques.

IV-2 Communication bidirectionnelle ESP32 ↔ Cloud

- **Client** : l'ESP32 embarque *esp-websocket-client* ou le SDK Arduino « WebSockets-Client ».
- **Etablissement** : l'ESP32 signe la requête (SigV4) ou présente son certificat X.509, puis ouvre un tunnel WSS 443 vers *wss://xxxxx.execute-api.* ou *wss://endpoint-ats.iot..*
- **Echanges** :
 - *Upstream* : capteurs (JSON) → route publishMessage ; Lambda stocke ou publie dans un topic.
 - *Downstream* : Lambda ou Rule IoT envoie LED_ON, buzzer=90 dB → API Gateway pousse directement à l'ESP32 par son ID.
- **Latence typique** : < 100 ms intra-Région, car aucune reconnexion ni polling. Des développeurs ont validé ce flux sur ESP32 + IoT Core avec WSS ; la bibliothèque supporte maintenant la signature SigV4 et la reconnexion automatique

IV-3 Pourquoi WebSocket plutôt que HTTP (long-polling) ou MQTT 1883 ?

- **WebSocket vs HTTP** : on évite le *polling* qui gaspille batterie ; un seul handshake TLS pour des centaines d'évènements.
- **WebSocket vs MQTT natif** : même sémantique publish / subscribe possible via IoT Core mais sur port 443 => passage firewall garanti ; même canal pour navigateur et microcontrôleur → simplifie l'architecture.

En résumé, la pile *ESP32* → *WebSocket* → *API Gateway* / *IoT Core* → *Lambda* combine **temps-réel**, **sécurité TLS**, et **élasticité serverless** — idéale pour une maison intelligente où les événements doivent être poussés instantanément, sans gérer d'infrastructure ni se heurter aux restrictions réseau.

V- Développement logiciel embarqué (ESP32/Arduino)

V-1 Langage et Environnement

- **Arduino framework (C/C++) sur ESP-IDF caché;** support natif Wi-Fi, ADC, interruptions.
- **Structure classique :**

```
void setup() { /* init */ }
```

```
void loop() { /* tâche principale */ }
```

- **Librairies clés :**

fonctions	Librairie	Observations
Connexion Wifi	<WiFi.h>	Gère STA + WPA2
WebSocket TLS	<WebSocketsClient.h>	Compatible AWS API GW / IoT Core (port 443)
JSON Leger	<ArduinoJson.h>	Sérialisation rapide < 2 kB RAM

V-2 Gestion des Interruptions

Capteur	Broche	Type declencheur	ISR (Interrupt Service Routine)
Tactile	GPIO14	RISING(Appui)	Allumer/éteindre LED 1
Detecteur Son(U3)	GPIOA2	CHANGE	Capturer pic sonore

V-2 Gestion des Interruptions

Capteurs/Evenements	Methode	Implementations
Tuoch Sensor (capacitif)	Interruption matérielle sur les pins <i>T0-T9</i> grâce à <code>touchAttachInterrupt()</code>	Seuil réglable : <code>touch_pad_config(TxPin, threshold)</code> ; ISR minimaliste → déclenche un flag ou poste à une queue FreeRTOS
Capteur sonore analogique	Seuil logiciel dans une tâche haute priorité (ADC 5 kbps) ou DMA + IIR pour éviter les faux positifs	Quand amplitude > <code>SOUND_THRESHOLD</code> , envoie un événement par <code>xQueueSendFromISR()</code>
Capteur ultrasonique	Timer + interrupt sur <i>echo</i> pour mesurer la durée d'impulsion	Utilisation de <code>pent</code> ou <code>interruptIn</code> pour éviter le <code>pulseIn()</code> bloquant
Watchdog / sécurité	<code>esp_task_wdt_reset()</code> dans la boucle principale	Assure le redemarrage en cas de blocage sur une longue durée.

VI- Intégration et tests

VI-1 Méthodologie et tests unitaires

L'intégration s'est déroulée selon une approche modulaire avec validation de chaque composant individuellement.

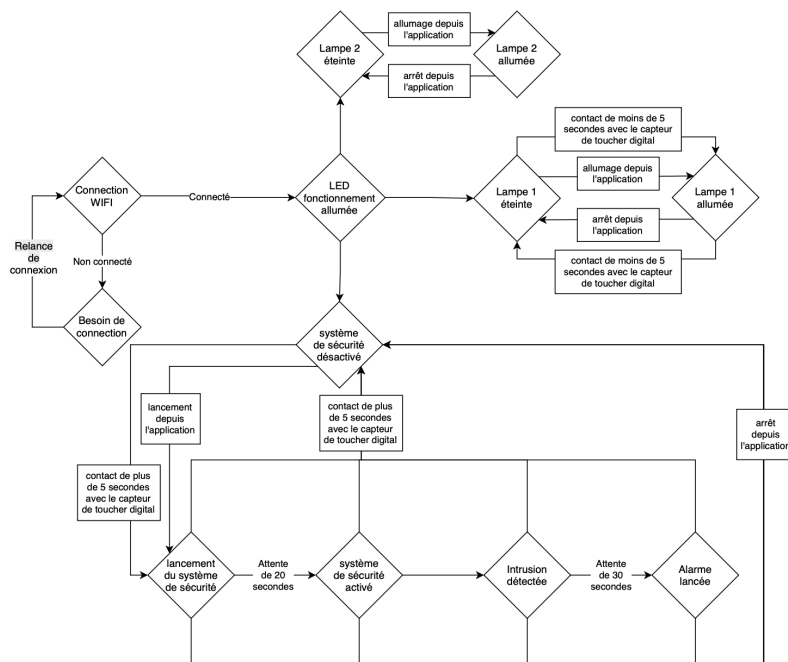
Tests des composants : Le capteur tactile a été validé pour la détection d'appui court/long avec anti-rebond (50ms). Le capteur sonore utilise un moyennage sur 50 échantillons avec calibration automatique des seuils. Le haut-parleur supporte les tonalités 200Hz-1600Hz pour alertes. Le capteur ultrasonique couvre la plage 2-500cm avec précision $\pm 2\text{cm}$.

Tests système : L'architecture matérielle ne présente aucun conflit de pins avec alimentation stable 3.3V. La communication WebSocket TLS vers AWS fonctionne avec reconnexion automatique.

VI-2 Tests fonctionnels et performance

Scénarios sécurité : Les cinq états (désarmé, armement 20s, armé, intrusion 30s, alarme) fonctionnent correctement avec transitions temporelles respectées. Tests multi-utilisateurs validés sur 5 connexions simultanées.

Métriques : Latence détection <100ms, commande distante ~250ms, contrôle physique <50ms. Fiabilité >99% sur 72h, autonomie théorique >48h sur batterie 2000mAh.



VII- Limites et perspectives

VII-1 Limitations actuelles

Contraintes audio et IA : Le capteur sonore DFR0034 est un simple détecteur de niveau qui ne peut capturer de parole intelligible. Le haut-parleur FIT0449 ne restitue pas de voix synthétisée claire. Ces limitations empêchent l'intégration d'IA générative (reconnaissance vocale, assistant virtuel, synthèse vocale).

Sécurité : L'interface de contrôle manque d'authentification - toute personne connaissant l'URL WebSocket peut contrôler le système. Absence de chiffrement end-to-end, gestion des droits utilisateurs, et protection contre le déni de service.

Matériel : Portée WiFi limitée, capteurs sensibles aux variations environnementales, autonomie batterie restreinte, pas de redondance.

VII-2 Perspectives d'amélioration

Évolution IA : Intégration micro numérique (INMP441), enceinte haute qualité, processeur ESP32-S3 avec support IA, services cloud AWS Transcribe/Polly.

Sécurité : Authentification JWT, chiffrement AES-256, gestion rôles utilisateurs, audit trail.

Extensions : Capteurs additionnels (caméra, fumée), géolocalisation pour armement automatique, compatibilité Matter/Thread, machine learning anti-fausse alarmes.

VIII- Conclusion

VIII-1 Bilan des objectifs

Ce projet a pleinement atteint ses objectifs de déploiement d'une architecture domotique fonctionnelle, économique (<80\$ CAD) et reproductible. L'intégration ESP32 + capteurs multiples + communication WebSocket AWS démontre la viabilité des solutions IoT open-source.

Réussites : Architecture modulaire avec PCB personnalisé, communication temps-réel 250ms de latence, système sécurité 5 états opérationnel, interface React moderne, stabilité 72h continue.

VIII-2 Compétences acquises

Le projet a développé des compétences transversales malgré le temps d'appropriation collaborative nécessaire :

- **Électronique embarquée** : Câblage multi-capteurs, gestion interruptions (niveau intermédiaire)
- **Programmation embarquée** : Arduino Framework, tâches non-bloquantes (niveau avancé)
- **Architecture cloud** : WebSocket serverless, AWS Lambda/DynamoDB (niveau intermédiaire)
- **Méthodologie projet** : Prototypage itératif, tests modulaires (niveau avancé)
- **Cybersécurité IoT** : Communication TLS, bonnes pratiques (niveau débutant)

VIII-3 Impact et recommandations

Valeur démonstrative : Ce projet illustre l'accessibilité des architectures IoT modernes avec composants standard et cloud managé. La modularité permet extensions progressives et scalabilité horizontale.

Conclusion finale : Ce projet montre qu'il est possible de créer un système de maison intelligente fonctionnel avec un budget limité et des connaissances de base en électronique. Avec 80\$ et quelques semaines de travail, nous avons obtenu un système qui rivalise avec des solutions commerciales beaucoup plus chères. Les compétences techniques acquises nous permettront d'aborder des projets IoT plus ambitieux à l'avenir.

Bibliographie

- docs.aws.amazon.com
- <https://www.espressif.com/en/support/documents/technical-documents>
- <https://github.com/aws/aws-iot-device-sdk-embedded-C>
- <https://docs.aws.amazon.com/iot/latest/developerguide/protocols.html>
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api-overview.html>
- <https://www.arduino.cc/reference/en/>
- <https://github.com/Links2004/arduinoWebSockets>
- <https://arduinojson.org/>
- <https://randomnerdtutorials.com/>
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api-authorization.html>
- https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_tls.html
- Cours de IoT